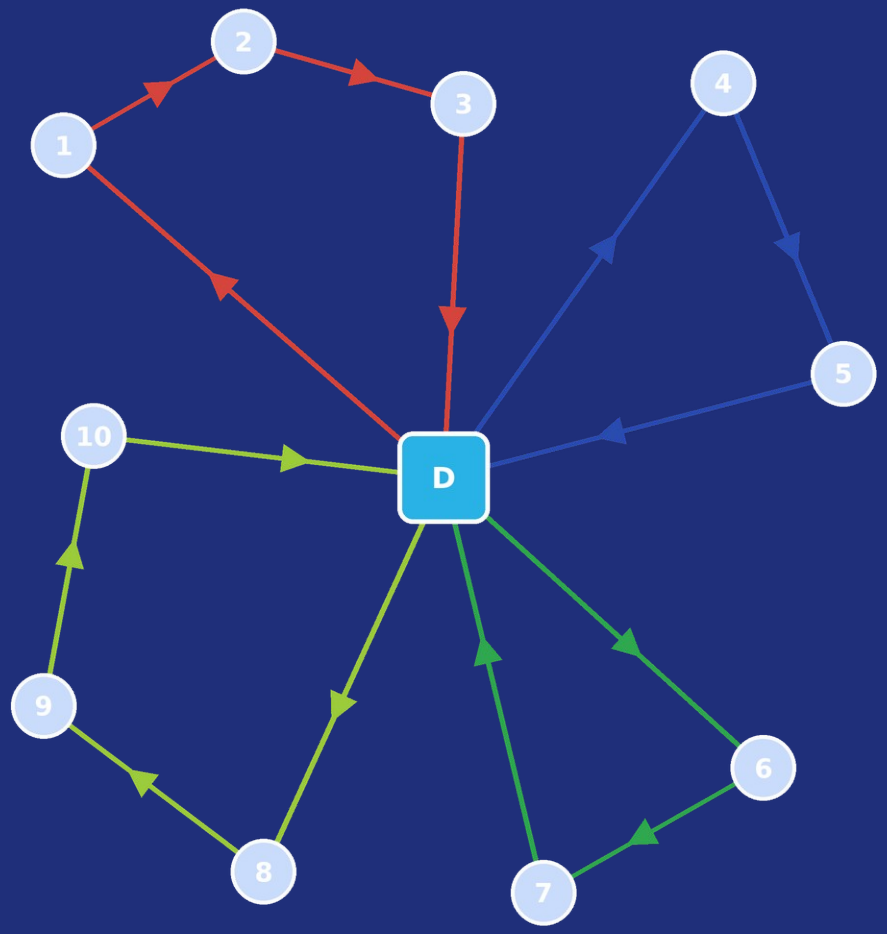
Ruteo de vehículos

Modelos matemáticos de las variantes del VRP y algoritmos heurísticos y metaheurísticos

Juan Sebastián Salas Lugo
Distribución y Transporte · UTB
Profesor: Jaime Acevedo Chedid



AGENDA

Qué vamos a ver

A

Lo visto en clase

TSP, mTSP, CVRP y VRPTW.

B

Otros modelos matemáticos

Variantes con su modelo, código GAMS y pruebas.

C

Heurísticas y metaheurísticas

Cómo se resuelven en la práctica, con un ejemplo paso a paso.

D

Ejemplos resueltos

Casos de Cali y la herramienta con 27 variantes.

POR QUÉ IMPORTA EL VRP

El transporte es la palanca económica de la cadena

6–7 %

del precio final de los productos corresponde a costos de transporte

40–60 %

de los costos logísticos totales los generan las decisiones de transporte

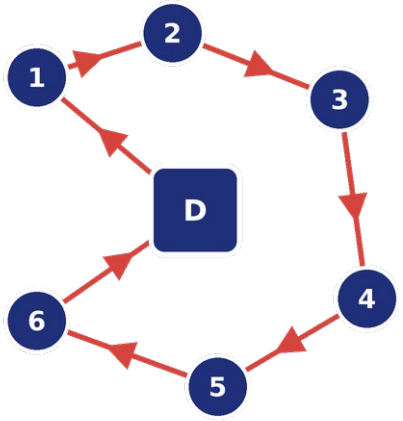
8 %

del PIB de Estados Unidos (2021) está anclado a la logística

El VRP no es solo un desafío matemático: es la principal palanca económica para reducir el costo de los bienes.

Repaso: de TSP a VRPTW

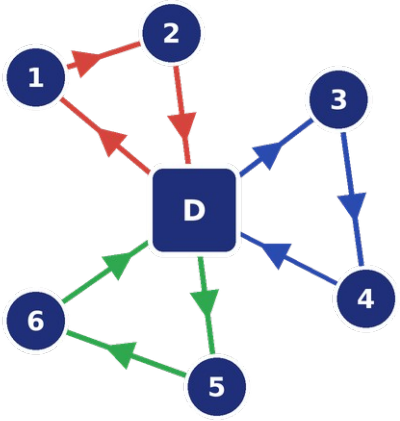
TSP



Un viajante, n clientes

Entrar y salir una vez · MTZ

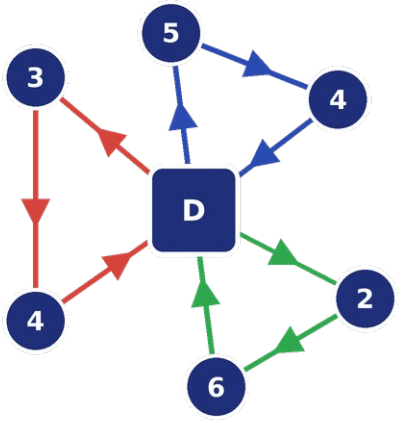
mTSP



m viajantes desde un depósito

$\Sigma x_{0j} = m \cdot \Sigma x_{j0} = m \cdot$ MTZ con p

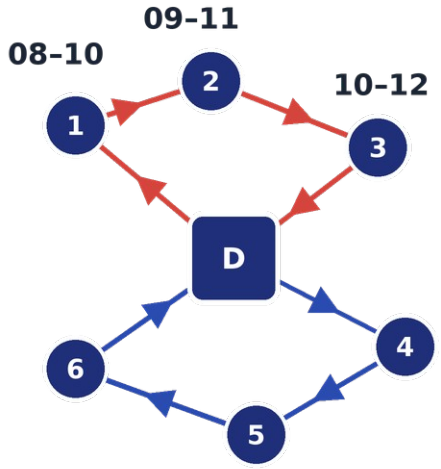
CVRP



Flota con capacidad y demanda

$\Sigma\ d_j\ x_{jik} \le K_{0k}$

VRPTW



Ventana de tiempo $[a_i, b_i]$

$s_{ik} + serv_i + t_{ij} - s_{jk} + M\ x_{ijk} \le M$

Notación común para las variantes

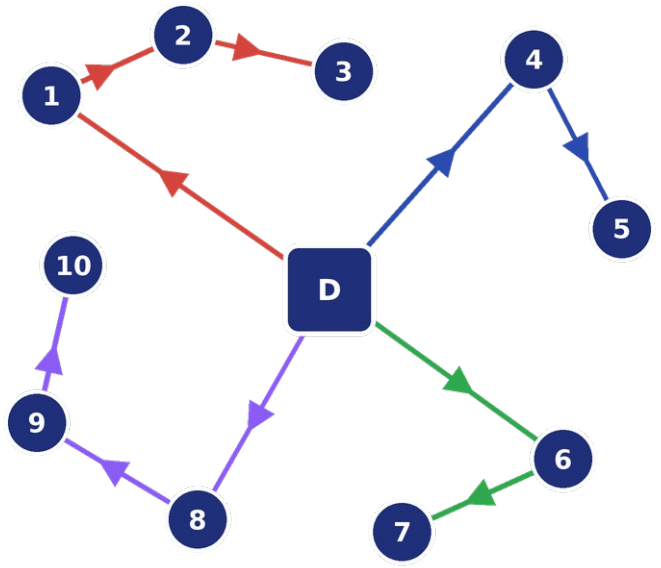
$N = \{1,\dots,n\}$	clientes
$N_0 = N \cup \{0\}$	clientes y depósito
K	vehículos $k = 1,\dots,K$
c_{ij}	costo (distancia) de i a j
t_{ij}	tiempo de viaje de i a j
d_j	demanda del cliente j
Q_k	capacidad del vehículo k
$x_{ijk} \in \{0,1\}$	k va de i a j

Núcleo de casi todos los modelos

- Objetivo: $Z = \Sigma_k \Sigma_i \Sigma_j c_{ij} x_{ijk}$
- Cada cliente se atiende una vez
- El vehículo que entra es el que sale
- Cada vehículo sale del depósito a lo más una vez
- Eliminación de sub-tours

OVRP: ruteo abierto (sin retorno al depósito)

Cómo se ve



$$\text{Min } Z = \sum_k \sum_i \sum_j c_{ij}\, x_{ijk} \qquad \text{(sin arcos al depósito)}$$

$$(1)\ \sum_k \sum_i x_{ijk} = 1 \qquad \forall\, j \in N$$

$$(2)\ \sum_l x_{jlk} \le \sum_i x_{ijk} \qquad \forall\, j, k \quad \text{(puede terminar en } j\text{)}$$

$$(3)\ \sum_j x_{0jk} \le 1 \qquad \forall\, k$$

$$(4)\ \sum_j d_j \sum_i x_{ijk} \le Q_k \qquad \forall\, k$$

$$(5)\ u_i - u_j + n\, x_{ijk} \le n - 1 \qquad \text{sub-tours (MTZ)}$$

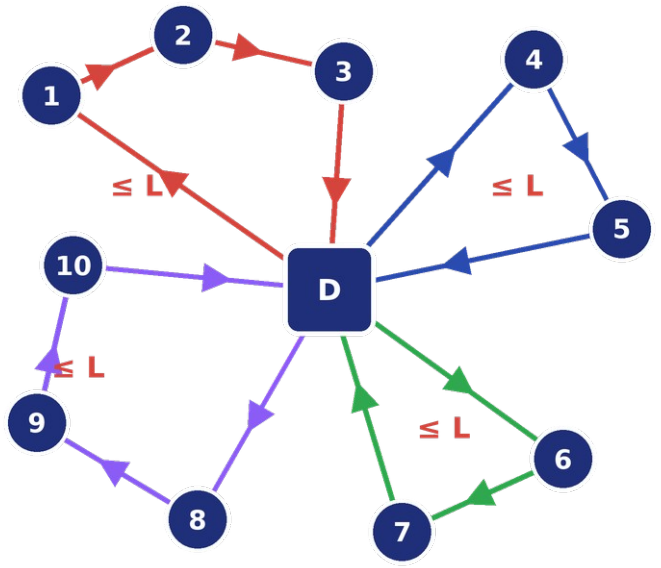
Qué cambia: El vehículo termina en su último cliente: el flujo (4) del CVRP pasa a desigualdad.

Aplicación real · transportistas externos y rutas escolares

Repartidores subcontratados que no vuelven a bodega; buses escolares que terminan en el colegio o recogen personal que termina en su casa.

DCVRP: capacidad y distancia máxima por ruta

Cómo se ve



$$\text{Min } Z = \sum_k \sum_i \sum_j c_{ij}\, x_{ijk}$$

Modelo CVRP + límite de distancia por vehículo:

$$(7)\ \sum_i \sum_j c_{ij}\, x_{ijk} \le L \qquad \forall\, k \in K$$

L = kilómetros (o horas) máximos por ruta

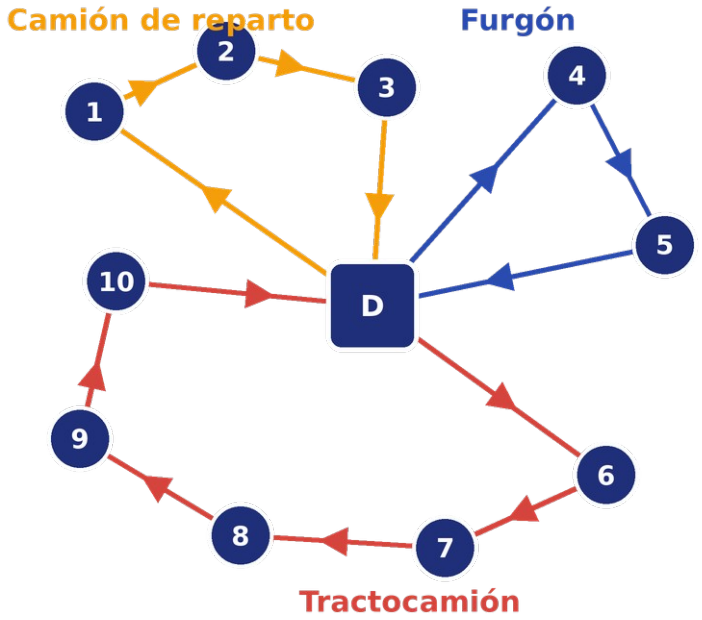
Qué cambia: Cada ruta tiene un tope de kilómetros o de jornada.

Aplicación real · última milla con jornada máxima

Mensajería urbana: ningún conductor supera 8 horas ni 120 km, para reducir fatiga, accidentes y consumo de combustible.

HFVRP: flota heterogénea

Cómo se ve



Tipo t: capacidad Q_t, costo fijo f_t, costo por km α_t, disponibles R_t

$$\text{Min } Z = \sum_t \sum_{k \in K_t} \left[f_t \sum_j x_{0jk} + \alpha_t \sum_i \sum_j c_{ij} x_{ijk} \right]$$

$$(6') \quad \sum_j d_j \sum_i x_{ijk} \le Q_t \qquad \forall\, k \in K_t$$

$$(8) \quad |K_t| \le R_t \qquad \forall\, t$$

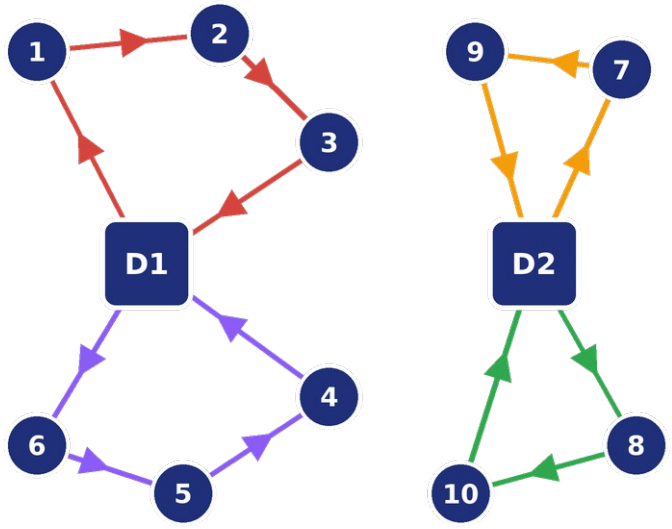
Qué cambia: Cada tipo de vehículo tiene su capacidad y su costo; el modelo decide cuántos usar.

Aplicación real · logística urbana con flota mixta

Un distribuidor usa camiones de reparto en barrios y un tractocamión para pedidos grandes; el modelo reparte la carga según costo fijo y costo por km.

MDVRP: múltiples depósitos

Cómo se ve



D = depósitos; Kd = vehículos del depósito d

Min Z = Σd Σk∈Kd Σi Σj ci j xi j k

(1) Σk Σi xi j k = 1 ∀ j ∈ N
(2) Σj x(d,j,k) ≤ 1 ∀ d, k ∈ Kd sale de d
(3) Σj x(d,j,k) = Σi x(i,d,k) regresa a d
(4) Σj dj Σi xi j k ≤ Q ∀ k ∈ Kd

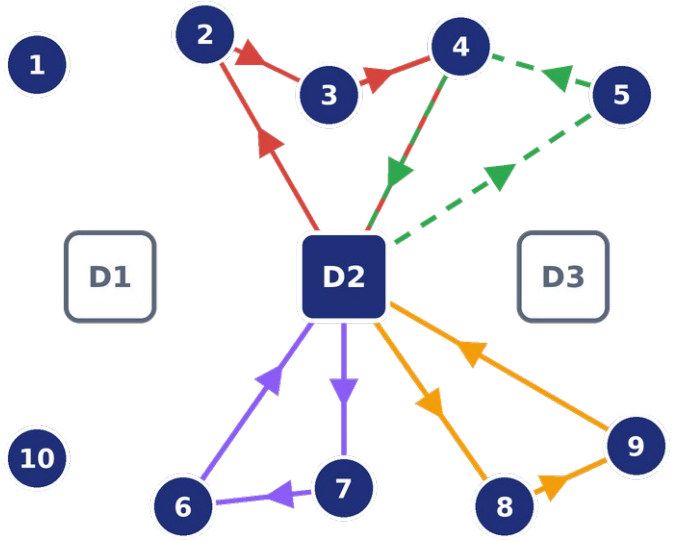
Qué cambia: Cada ruta empieza y termina en el mismo depósito; hay que asignar clientes a depósitos.

Aplicación real · cadenas con varios centros de distribución

Una cadena de supermercados atiende sus tiendas desde dos bodegas regionales; cada tienda se asigna a la bodega que reduce el costo total.

LRP: localización y ruteo a la vez

Cómo se ve



$y_d \in \{0,1\}$: abrir d; f_d costo fijo; W_d capacidad

$$\text{Min } Z = \Sigma_d f_d y_d + \Sigma_k \Sigma_i \Sigma_j c_{ij} x_{ijk}$$

(1) $\Sigma_j x(d,j,k) \le y_d \quad \forall d, k$ solo si está abierto

(2) $\Sigma_j d_j z_{jd} \le W_d y_d \quad \forall d$ capacidad

(3) $\Sigma_d z_{jd} = 1 \quad \forall j$ un depósito por cliente

\+ restricciones de ruteo (CVRP)

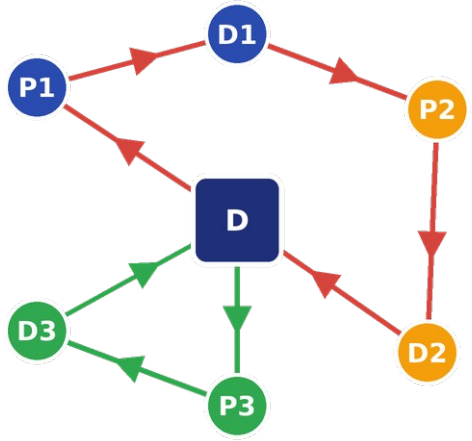
Qué cambia: Además de rutear, se decide qué depósitos abrir (cuadros llenos = abiertos).

Aplicación real · dónde ubicar el centro de distribución

Una cadena de tiendas (ARA, Cali) decide dónde poner su centro de distribución para que las rutas de los supervisores sean lo más cortas posibles.

Recogidas y entregas, y entrega dividida

VRPPD · recogida y entrega

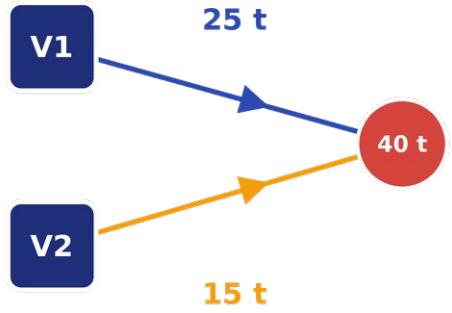


$$\sum_j x_{ijk} = \sum_j x_{(n+i),j,k}$$
$$s_{ik} + serv_i + t_{i,n+i} \le s_{(n+i),k}$$
$$L_{jk} \ge L_{ik} + q_j - M(1 - x_{ijk})$$
$$0 \le L_{ik} \le Q_k$$

P = recoger, D = entregar. Mismo vehículo y la recogida antes de la entrega.

Aplicación: mensajería, taxis compartidos, traslado de pacientes.

SDVRP · entrega dividida



$$y_{ik} \ge 0 \quad \text{cantidad entregada}$$
$$\sum_k y_{ik} = d_i$$
$$y_{ik} \le d_i \sum_j x_{ijk}$$
$$\sum_i y_{ik} \le Q_k$$

Un cliente con más demanda que un vehículo se atiende entre varios.

Aplicación: carga a granel, cemento o grano, cuando un pedido supera un camión.

Otras variantes: qué cambia y dónde se usan

Variante	Qué cambia en el modelo	Aplicación
TDVRP · tráfico	$t_{ij}(s)$ depende de la hora de salida (lineal por tramos, FIFO)	Reparto urbano en hora pico
2E-VRP · dos niveles	Depósito → satélites → clientes; flujo entre niveles	Centro de consolidación urbano
VRPPC · flota propia y tercerizada	$z_j = 1$ si se subcontrata el cliente j, con costo g_j	Picos de demanda en temporada
COMVRP · abiertas y cerradas	Cada vehículo elige si regresa; $c_{i0} = 0$ si es abierta	Flota mixta propia y contratada
SVRP · estocástico	$P(\Sigma d_j \leq Q_k) \geq 1 - \alpha$, o modelo de recurso	Recolección de residuos
VRPD · con drones	Lanzamiento y recogida en el camión; autonomía y sincronía	Zonas rurales o de difícil acceso

27 variantes de VRP en la herramienta (1/2)

#	Variante	Familia	Qué modela
1	TSP	Un vehículo	Un viajante, ruta cerrada.
2	TSP abierto	Un vehículo	Un viajante que no regresa al origen.
3	m-TSP	Varios viajantes	m viajantes desde un depósito, cada uno visita al menos un cliente.
4	m-TSP balanceado	Varios viajantes	Todas las rutas con igual número de clientes (±1).
5	Open m-TSP	Varios viajantes	Rutas que no regresan al depósito.
6	Open m-TSP balanceado	Varios viajantes	Rutas abiertas y equilibradas.
7	m-TSP min-max	Varios viajantes	Minimiza la ruta más larga.
8	CVRP	Capacidad	Vehículos con capacidad Q; cada cliente tiene demanda.
9	OVRP (ruteo abierto)	Capacidad	CVRP en el que las rutas no regresan al depósito.
10	DCVRP (distancia máxima)	Capacidad	CVRP con distancia máxima L por ruta.
11	CVRP con máximo de paradas	Capacidad	CVRP con máximo de clientes por ruta.
12	CVRP con mínima flota	Capacidad	Minimiza primero la cantidad de vehículos usados.
13	CVRP balanceado	Capacidad	CVRP con rutas de igual número de clientes.
14	VRPTW (ventanas de tiempo)	Tiempo	CVRP con ventana horaria en cada cliente.

27 variantes de VRP en la herramienta (2/2)

#	Variante	Familia	Qué modela
15	OVRPTW	Tiempo	VRPTW con rutas abiertas.
16	HFVRP (flota heterogénea)	Flota	Tipos de vehículo con capacidad y costo por km distintos.
17	FSMVRP (tamaño y mezcla de flota)	Flota	Flota mixta con costo fijo por vehículo usado.
18	HFVRPTW	Flota	Flota heterogénea con ventanas de tiempo.
19	MDVRP (varios depósitos)	Depósitos	Cada ruta sale y vuelve al mismo depósito.
20	MDVRPTW	Depósitos	Varios depósitos con ventanas de tiempo.
21	LRP (localización y ruteo)	Depósitos	Decide qué depósitos abrir y las rutas.
22	VRPB (backhauls)	Cargas	Entregas primero, luego recogidas en el camino de regreso.
23	VRPSPD (recoge y entrega a la vez)	Cargas	Cada cliente recibe y entrega mercancía en la misma visita.
24	VRPPD (recogida y entrega por pares)	Cargas	Pares origen-destino; misma ruta y la recogida primero.
25	SDVRP (entrega dividida)	Cargas	La demanda de un cliente puede atenderse entre varios vehículos.
26	VRP con beneficios	Selectivos	Visita solo los clientes rentables: maximiza premio − costo.
27	Orienteering / TOP	Selectivos	Maximiza el premio recogido con distancia máxima por ruta.

GAMS · modelo base del CVRP

```
* Modelo CVRP
Set arco(i,j,k) 'arcos permitidos';
arco(i,j,k)$(ord(i)<>ord(j) and (c(i) or
    kd(k,i)) and (c(j) or kd(k,j))) = yes;
Parameter cr(i,j,k) 'costo del arco';
cr(i,j,k)$arco(i,j,k) = cu(k)*dist(i,j);
Binary Variables x(i,j,k), y(i,k), v(k);
Positive Variable u(i) 'orden de visita
    (MTZ)';
Variable z;
Equations fo, visita(i), entra(i,k),
    sale(i,k), parte(k), vuelve(k), usa(i,k),
    dos(i,j,k), mtz(i,j);
x.fx(i,j,k)$(not arco(i,j,k)) = 0;
fo.. z =e= sum((i,j,k)$arco(i,j,k),
    cr(i,j,k)*x(i,j,k));
visita(c).. sum(k, y(c,k)) =e= 1;
```

```
entra(c,k).. sum(j$arco(j,c,k), x(j,c,k)) =e=
    y(c,k);
sale(c,k).. sum(j$arco(c,j,k), x(c,j,k)) =e=
    y(c,k);
parte(k).. sum((i,j)$(kd(k,i) and c(j)),
    x(i,j,k)) =e= v(k);
vuelve(k).. sum((i,j)$(c(i) and kd(k,j)),
    x(i,j,k)) =e= v(k);
usa(c,k).. y(c,k) =l= v(k);
dos(i,j,k)$(c(i) and c(j) and ord(i)<ord(j)).
    x(i,j,k) + x(j,i,k) =l= y(i,k);
mtz(i,j)$(c(i) and c(j) and ord(i)<>ord(j)).
    u(i) - u(j) + card(c)*sum(k, x(i,j,k)) =l=
    card(c) - 1;
u.lo(c) = 1; u.up(c) = card(c);
Equation cap(k);
cap(k).. sum(c, w(c)*y(c,k)) =l= Q(k)*v(k);
```

Prueba (7 nodos): CVRP: MILP 82.600 = fuerza bruta 82.600 ✓

GAMS · rutas abiertas, distancia máxima y balanceo

```
* Se agrega en OVRP (ruteo abierto)
cr(i,j,k)$(arco(i,j,k) and kd(k,j)) = 0;   *
    ruta abierta: regresar no cuesta
* Se agrega en DCVRP (distancia máxima)
Equation lim(k);
lim(k).. sum((i,j)$arco(i,j,k),
    (cr(i,j,k)/cu(k))*x(i,j,k)) =l=
    Lmax(k)*v(k);
```

```
* Se agrega en m-TSP balanceado
v.lo(k) = 1;
Equations bal1(k), bal2(k);
bal1(k).. sum(c, y(c,k)) =g=
    floor(card(c)/card(k))*v(k);
bal2(k).. sum(c, y(c,k)) =l=
    ceil(card(c)/card(k))*v(k);
```

Prueba (7 nodos): OVRP (ruteo abierto): MILP 54.500 = fuerza bruta 54.500 ✓ · DCVRP (distancia máxima): MILP 147.000 = fuerza bruta 147.000 ✓ · m-TSP balanceado: MILP 158.600 = fuerza bruta 158.600 ✓

GAMS · ventanas de tiempo y flota con costo fijo

```
* Se agrega en VRPTW (ventanas de tiempo)
Positive Variable t(i) 'hora de llegada';
Equations tw0(i,j,k), tw(i,j,k);
t.lo(c) = ae(c); t.up(c) = be(c);
tw0(i,j,k)$(kd(k,i) and c(j)).. t(j) =g= t0
    + dist(i,j)/vel(k) - Mt*(1 - x(i,j,k));
tw(i,j,k)$(c(i) and c(j) and
    ord(i)<>ord(j)).. t(j) =g= t(i) + srv +
    dist(i,j)/vel(k) - Mt*(1 - x(i,j,k));
* Se agrega en FSMVRP (tamaño y mezcla de
flota)
fo.. z =e= sum((i,j,k)$arco(i,j,k),
    cr(i,j,k)*x(i,j,k)) + sum(k,
    fx(k)*v(k));
```

Prueba (7 nodos): VRPTW (ventanas de tiempo): MILP 147.000 = fuerza bruta 147.000 ✓ · FSMVRP (tamaño y mezcla de flota): MILP 113.100 = fuerza bruta 113.100 ✓

GAMS · varios depósitos y localización-ruteo

```
* Se agrega en MDVRP (varios depósitos)
* Se agrega en LRP (localización y ruteo)
Binary Variable op(i);
fo.. z =e= sum((i,j,k)$arco(i,j,k),
    cr(i,j,k)*x(i,j,k)) + sum(dep,
    fa(dep)*op(dep));
Equation abre(k);
abre(k).. v(k) =l= sum(i$kd(k,i), op(i));
```

Prueba (7 nodos): MDVRP (varios depósitos): MILP 101.600 = fuerza bruta 101.600 ✓ · LRP (localización y ruteo): MILP 220.100 = fuerza bruta 220.100 ✓

GAMS · backhauls y recoger-entregar simultáneo

```
* Se agrega en VRPB (backhauls)
arco(i,j,k)$(bh(i) and lh(j)) = no;
arco(i,j,k)$(kd(k,i) and bh(j)) = no;
Equation capb(k);
capb(k).. sum(c, r(c)*y(c,k)) =l= Q(k)*v(k);
* Se agrega en VRPSPD (recoge y entrega a la
vez)
Positive Variable l(i) 'carga al salir del
    cliente';
Equations carga(i,j), carga0(i,j,k),
    lcap(i);
Scalar Ml 'big-M de carga' /16/;
carga(i,j)$(c(i) and c(j) and
    ord(i)<>ord(j)).. l(j) =g= l(i) + r(j) -
    w(j) - Ml*(1 - sum(k, x(i,j,k)));
carga0(i,j,k)$(kd(k,i) and c(j)).. l(j) =g=
    sum(c, w(c)*y(c,k)) + r(j) - w(j) -
    Ml*(1 - x(i,j,k));
lcap(c).. l(c) =l= sum(k, Q(k)*y(c,k));
```

Prueba (7 nodos): VRPB (backhauls): MILP 79.700 = fuerza bruta 79.700 ✓ · VRPSPD (recoge y entrega a la vez): MILP 103.100 = fuerza bruta 103.100 ✓

GAMS · recogida-entrega por pares y entrega dividida

```
* Se agrega en VRPPD (recogida y entrega por
pares)
Positive Variable l(i) 'carga al salir del
    cliente';
Equations carga(i,j), carga0(i,j,k),
    lcap(i);
Scalar Ml 'big-M de carga' /9/;
carga(i,j)$(c(i) and c(j) and
    ord(i)<>ord(j)).. l(j) =g= l(i) + r(j) -
    w(j) - Ml*(1 - sum(k, x(i,j,k)));
carga0(i,j,k)$(kd(k,i) and c(j)).. l(j) =g=
    r(j) - w(j) - Ml*(1 - x(i,j,k));
lcap(c).. l(c) =l= sum(k, Q(k)*y(c,k));
Equations mismo(i,j,k), prec(i,j);
mismo(i,j,k)$pd(i,j).. y(i,k) =e= y(j,k);
prec(i,j)$pd(i,j).. u(j) =g= u(i) + 1;
* Se agrega en SDVRP (entrega dividida)
Positive Variable q(i,k) 'entrega del
    vehículo k al cliente i';
Positive Variable u(i,k) 'orden de visita
    (MTZ por vehículo)';
Equations fo, visita(i), entra(i,k),
    sale(i,k), parte(k), vuelve(k),
    usa(i,k), dos(i,j,k), mtz(i,j,k);
visita(c).. sum(k, y(c,k)) =g= 1;
mtz(i,j,k)$(c(i) and c(j) and
    ord(i)<>ord(j)).. u(i,k) - u(j,k) +
    card(c)*x(i,j,k) =l= card(c) - 1;
u.lo(c,k) = 1; u.up(c,k) = card(c);
Equations ent(i,k), tot(i), capq(k);
ent(c,k).. q(c,k) =l= w(c)*y(c,k);
tot(c).. sum(k, q(c,k)) =e= w(c);
capq(k).. sum(c, q(c,k)) =l= Q(k)*v(k);
```

Prueba (7 nodos): VRPPD (recogida y entrega por pares): MILP 113.300 = fuerza bruta 113.300 ✓ · SDVRP (entrega dividida): MILP 147.000 = fuerza bruta n/a ✓

GAMS · problemas con beneficios y orienteering

```
* Se agrega en VRP con beneficios
fo.. z =e= sum((i,j,k)$arco(i,j,k),
    cr(i,j,k)*x(i,j,k)) - sum((c,k),
    prem(c)*y(c,k));
visita(c).. sum(k, y(c,k)) =l= 1;
* Se agrega en Orienteering / TOP
fo.. z =e= 0.0001*(sum((i,j,k)$arco(i,j,k),
    cr(i,j,k)*x(i,j,k))) - sum((c,k),
    prem(c)*y(c,k));
visita(c).. sum(k, y(c,k)) =l= 1;
Equation lim(k);
lim(k).. sum((i,j)$arco(i,j,k),
    (cr(i,j,k)/cu(k))*x(i,j,k)) =l=
    Lmax(k)*v(k);
```

Prueba (7 nodos): VRP con beneficios: MILP -127.680 = fuerza bruta -127.680 ✓ · Orienteering / TOP: MILP -254.506 = fuerza bruta -254.506 ✓

Prueba de los modelos (1/2)

Variante	MILP (HiGHS)	Fuerza bruta	Coincide	Mejor heur. (gap)	Algoritmos en el óptimo
TSP	132.10	132.10	✓	132.1 (0.0 %)	8 de 11
TSP abierto	59.70	59.70	✓	59.7 (0.0 %)	8 de 11
m-TSP	82.60	82.60	✓	82.6 (0.0 %)	5 de 11
m-TSP balanceado	158.60	158.60	✓	158.6 (0.0 %)	7 de 11
Open m-TSP	77.90	77.90	✓	77.9 (0.0 %)	7 de 11
Open m-TSP balanceado	76.50	76.50	✓	76.5 (0.0 %)	10 de 11
m-TSP min-max	59.50	59.50	✓	59.5 (0.0 %)	8 de 11
CVRP	82.60	82.60	✓	82.6 (0.0 %)	6 de 11
OVRP (ruteo abierto)	54.50	54.50	✓	54.5 (0.0 %)	5 de 11
DCVRP (distancia máxima)	147.00	147.00	✓	147.0 (0.0 %)	7 de 11
CVRP con máximo de paradas	82.60	82.60	✓	82.6 (0.0 %)	6 de 11
CVRP con mínima flota	81703.10	81703.10	✓	81703.1 (0.0 %)	10 de 11
CVRP balanceado	158.60	158.60	✓	158.6 (0.0 %)	7 de 11
VRPTW (ventanas de tiempo)	147.00	147.00	✓	147.0 (0.0 %)	7 de 11

Instancia de 7 nodos (6 clientes), 2–3 vehículos. La fuerza bruta enumera todas las asignaciones y rutas con el evaluador independiente de las heurísticas.

Prueba de los modelos (2/2)

Variante	MILP (HiGHS)	Fuerza bruta	Coincide	Mejor heur. (gap)	Algoritmos en el óptimo
OVRPTW	61.10	61.10	✓	61.1 (0.0 %)	8 de 11
HFVRP (flota heterogénea)	147.00	147.00	✓	147.0 (0.0 %)	7 de 11
FSMVRP (tamaño y mezcla de flota)	113.10	113.10	✓	113.1 (0.0 %)	10 de 11
HFVRPTW	144.50	144.50	✓	144.5 (0.0 %)	8 de 11
MDVRP (varios depósitos)	101.60	101.60	✓	101.6 (0.0 %)	5 de 11
MDVRPTW	118.40	118.40	✓	118.4 (0.0 %)	8 de 11
LRP (localización y ruteo)	220.10	220.10	✓	220.1 (0.0 %)	4 de 11
VRPB (backhauls)	79.70	79.70	✓	79.7 (0.0 %)	8 de 11
VRPSPD (recoge y entrega a la vez)	103.10	103.10	✓	103.1 (0.0 %)	10 de 11
VRPPD (recogida y entrega por pares)	113.30	113.30	✓	113.3 (0.0 %)	7 de 11
SDVRP (entrega dividida)	147.00	n/a (ver nota)	✓*	—	solo exacto
VRP con beneficios	-127.68	-127.68	✓	-127.7 (0.0 %)	5 de 11
Orienteering / TOP	-254.51	-254.51	✓	-254.5 (0.0 %)	10 de 11

✓ SDVRP: no tiene enumeración simple; se verificó que su costo es menor o igual al del CVRP y que se reparte toda la demanda.*

C

Heurísticas y metaheurísticas

¿Cómo se resuelven los VRP cuando son demasiado grandes para el método exacto?

¿Por qué no basta con el modelo exacto?

El VRP es NP-difícil

- Generaliza al TSP: el número de rutas crece factorialmente
- Los métodos exactos resuelven decenas o pocos cientos de clientes
- En la práctica hay miles de clientes y poco tiempo
- Una buena solución rápida vale más que el óptimo tarde

1960–70

Heurísticas simples

Intuición y mejora local

1970–80

Programación matemática

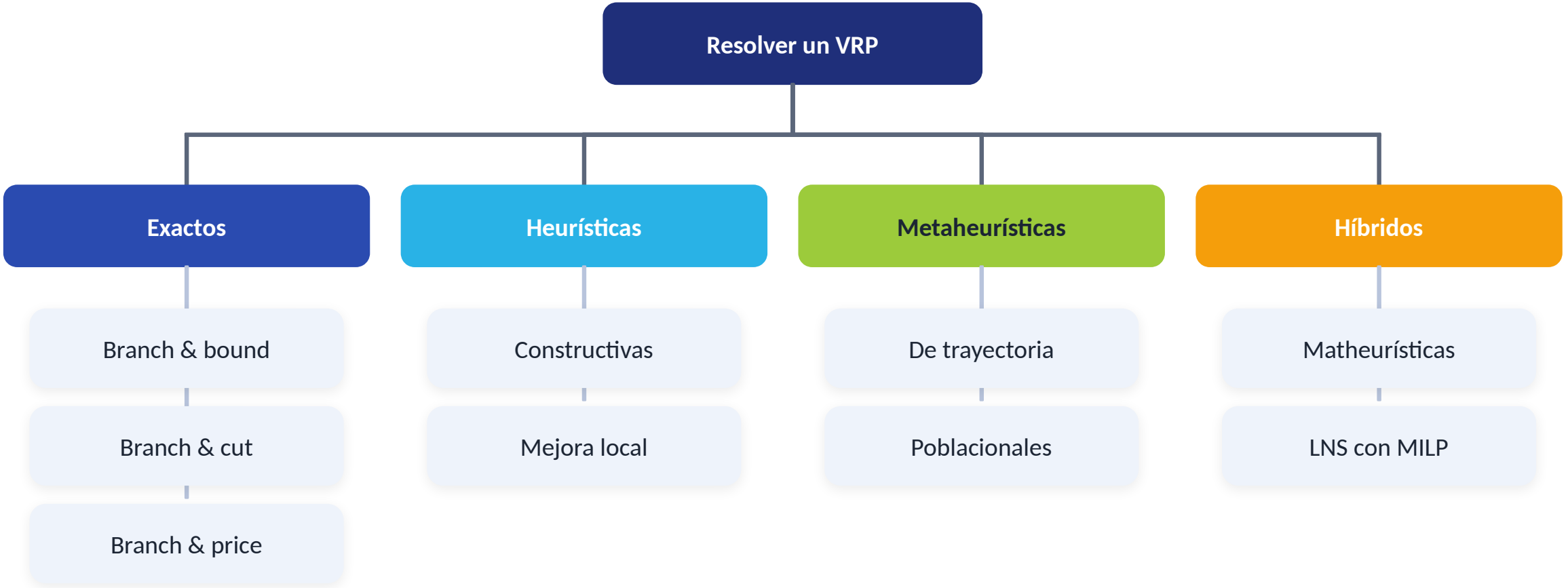
Asignación y agrupamiento (clustering)

1980–hoy

Metaheurísticas e IA

Búsqueda inteligente; soluciones guiadas por datos

Mapa de métodos de resolución



Trayectoria: tabú, recocido simulado, VNS · Poblacionales: algoritmo genético, colonia de hormigas

Cómo leer el pseudocódigo

Símbolos que usaremos

←	asignar un valor: s ← 5
`mientras …`	repetir mientras se cumpla una condición
`para cada …`	recorrer una lista o un conjunto
`si … si no`	decisión
`argmin`	el elemento que da el valor mínimo
∅	conjunto vacío
`c[i][j]`	costo (distancia) de ir de i a j
`d[j], Q`	demanda del cliente j; capacidad del vehículo

Cómo está organizada cada lámina

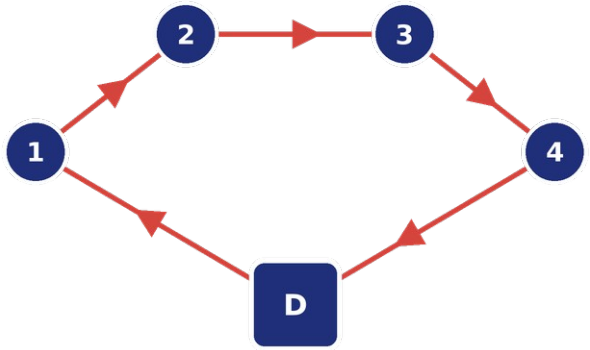
- A la izquierda, el pseudocódigo numerado.
- A la derecha, la idea en palabras y, cuando ayuda, un dibujo.
- Las palabras en azul son las instrucciones de control.
- Todos los algoritmos trabajan con el mismo núcleo: un depósito 0, clientes 1…n, costos c[i][j], demandas d[j] y capacidad Q.

Vecino más cercano: pseudocódigo

```
ruta ← [0]; carga ← 0; actual ← 0
mientras existan clientes sin visitar:
    C ← { j sin visitar : carga + d[j] ≤ Q }
    si C = ∅:
        volver al depósito; abrir ruta nueva
        carga ← 0; actual ← 0
    si no:
        j* ← argmin c[actual][j]   para j en C
        ir a j*; carga ← carga + d[j*]
        marcar j* como visitado; actual ← j*
cerrar la última ruta en el depósito
```

En palabras

- Desde donde está, va al cliente más cercano al que todavía le cabe la carga.
- Si no cabe ninguno, regresa al depósito y abre otra ruta.
- Muy rápido; el final de la ruta suele quedar largo.

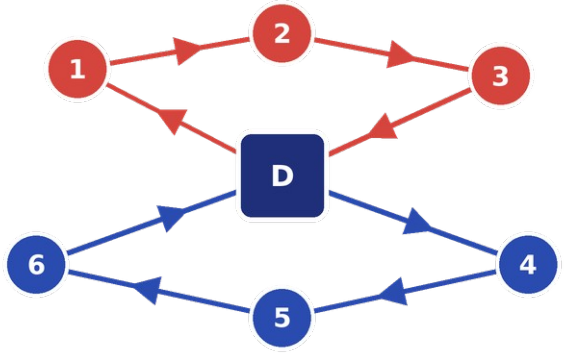


Barrido (sweep): pseudocódigo

```
para cada cliente j: θ[j] ← ángulo de j respecto al depósito
ordenar los clientes por θ creciente
ruta ← ∅; carga ← 0
para cada cliente j en ese orden:
    si carga + d[j] > Q:
        cerrar la ruta; abrir ruta nueva; carga ← 0
    agregar j a la ruta; carga ← carga + d[j]
para cada ruta:
    ordenar sus clientes resolviendo un TSP pequeño
```

En palabras

- Gira como un radar alrededor del depósito y va agrupando clientes.
- Cuando se llena el vehículo, cierra el grupo y empieza otro.
- Luego ordena cada grupo con un TSP.

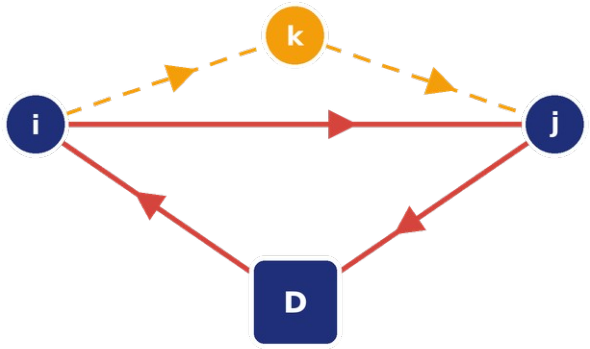


Inserción más barata: pseudocódigo

```
semilla s ← cliente más lejano; ruta ← [0, s, 0]
mientras haya clientes sin insertar:
    para cada cliente k y cada par (i, j) consecutivo:
        Δ ← c[i][k] + c[k][j] − c[i][j]
    escoger (k, posición) con menor Δ
        que respete capacidad Q (y ventanas de tiempo)
    si existe: insertar k en esa posición
    si no: abrir ruta nueva con una semilla
devolver las rutas
```

En palabras

- Insertar k entre i y j cuesta Δ = c(i,k) + c(k,j) − c(i,j).
- En cada paso se inserta el cliente que menos aumenta el costo.
- Admite ventanas de tiempo: solo se inserta donde se cumpla la franja.

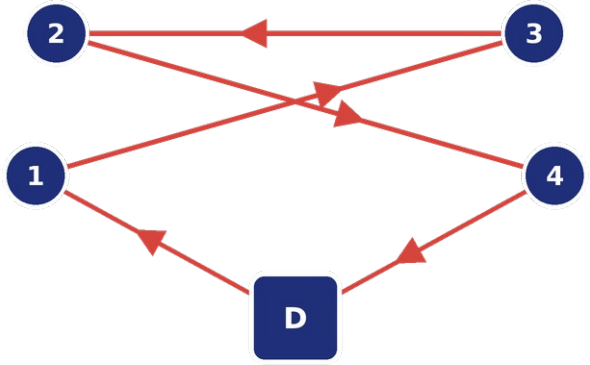


2-opt: pseudocódigo

```
ruta a = [a₀, a₁, …, aₙ, a₀]
mejora ← verdadero
mientras mejora:
    mejora ← falso
    para i = 0 … n−2:
        para j = i+2 … n:
            Δ ← c[aᵢ][aⱼ] + c[aᵢ₊₁][aⱼ₊₁]
                − c[aᵢ][aᵢ₊₁] − c[aⱼ][aⱼ₊₁]
            si Δ < 0:
                invertir el tramo aᵢ₊₁ … aⱼ
                mejora ← verdadero
devolver a
```

En palabras

- Quita dos arcos de la ruta y reconecta los extremos al revés.
- Si el costo baja (Δ < 0), se acepta el cambio.
- Se repite hasta que ningún cambio mejore: óptimo local.

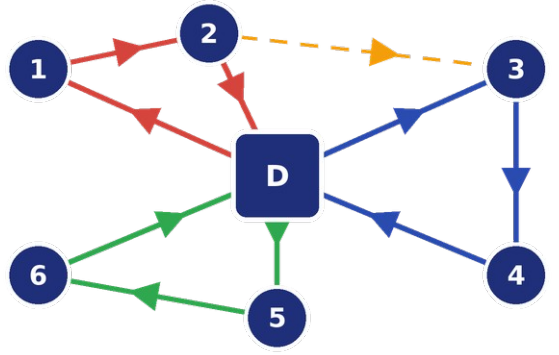


Relocate y swap entre rutas: pseudocódigo

```
repetir
    para cada cliente k de la ruta A:
        para cada posición p de otra ruta B:
            Δ ← costo(A sin k) + costo(B con k en p)
                − costo(A) − costo(B)
            si Δ < 0 y carga(B) + d[k] ≤ Q:
                mover k a la posición p de B
    (swap: intercambiar k de A con m de B
     si Δ < 0 y ambas rutas respetan Q)
hasta que ningún movimiento mejore
```

En palabras

- Relocate: pasa un cliente de una ruta a otra.
- Swap: intercambia dos clientes de rutas distintas.
- Solo se aceptan movimientos que bajan el costo y respetan la capacidad.



Metaheurísticas: cuatro ideas

1

Búsqueda tabú

Mejora local con memoria: lista tabú de movimientos recientes para no volver atrás.

Referencia clásica en VRP (Glover)

2

Recocido simulado

Acepta empeoramientos con probabilidad e^(−Δ/T) que baja al enfriar.

Simple de implementar

3

Algoritmo genético

Población de soluciones; cruce y mutación; sobrevive el más apto.

Explora mucho; flexible

4

Colonia de hormigas

Las hormigas dejan feromona en los buenos arcos; el rastro guía a las siguientes.

Natural en problemas de rutas

También: GRASP, VNS (cambio de vecindario) y ALNS (destruir y reparar), estado del arte en VRP con restricciones.

Búsqueda tabú: pseudocódigo

```
s ← solución inicial; mejor ← s; lista_tabú ← ∅
repetir hasta cumplir el número de iteraciones:
    V ← vecinos de s  (2-opt, relocate, swap)
    elegir s' ∈ V de menor costo tal que
        su movimiento no esté en lista_tabú
        o costo(s') < costo(mejor)     (aspiración)
    s ← s'
    agregar el movimiento a lista_tabú
        (olvidar el más antiguo si está llena)
    si costo(s) < costo(mejor): mejor ← s
devolver mejor
```

En palabras

- Siempre se pasa al mejor vecino, aunque empeore.
- La lista tabú prohíbe deshacer movimientos recientes, para no dar vueltas en círculo.
- La aspiración permite romper la prohibición si mejora el mejor global.

Parámetros

- Tamaño de la lista tabú
- Número de iteraciones

Búsqueda tabú paso a paso

Ejemplo: mejorar una ruta con 2-opt

Iter.	Movimiento	Costo	Lista tabú
0	Solución inicial	48,0	—
1	Invertir tramo 2–4	45,5	[2–4]
2	Invertir tramo 3–6	44,1	[2–4, 3–6]
3	Mejor vecino es peor (+1,2)	45,3	[3–6, 5–1]
4	Invertir tramo 1–5	43,0	[5–1, …]

Valores ilustrativos para explicar el mecanismo.

Ideas clave

- Siempre se pasa al mejor vecino, aunque empeore: así se sale de óptimos locales.
- La lista tabú prohíbe deshacer los últimos movimientos durante unas iteraciones.
- Criterio de aspiración: se permite un movimiento tabú si mejora la mejor solución conocida.
- Parámetros: tamaño de la lista y número de iteraciones.

Aplicación real · rutas de reparto diarias

Empresas de mensajería la usan para reajustar cada día las rutas de sus conductores tras agregar o cancelar pedidos.

Recocido simulado: pseudocódigo

```
s ← solución inicial; mejor ← s; T ← T₀
mientras T > T_min:
    repetir L veces:
        s' ← vecino aleatorio de s
        Δ ← costo(s') − costo(s)
        si Δ ≤ 0  o  aleatorio(0,1) < e^(−Δ/T):
            s ← s'
        si costo(s) < costo(mejor): mejor ← s
    T ← α · T                          (α ≈ 0,95)
devolver mejor
```

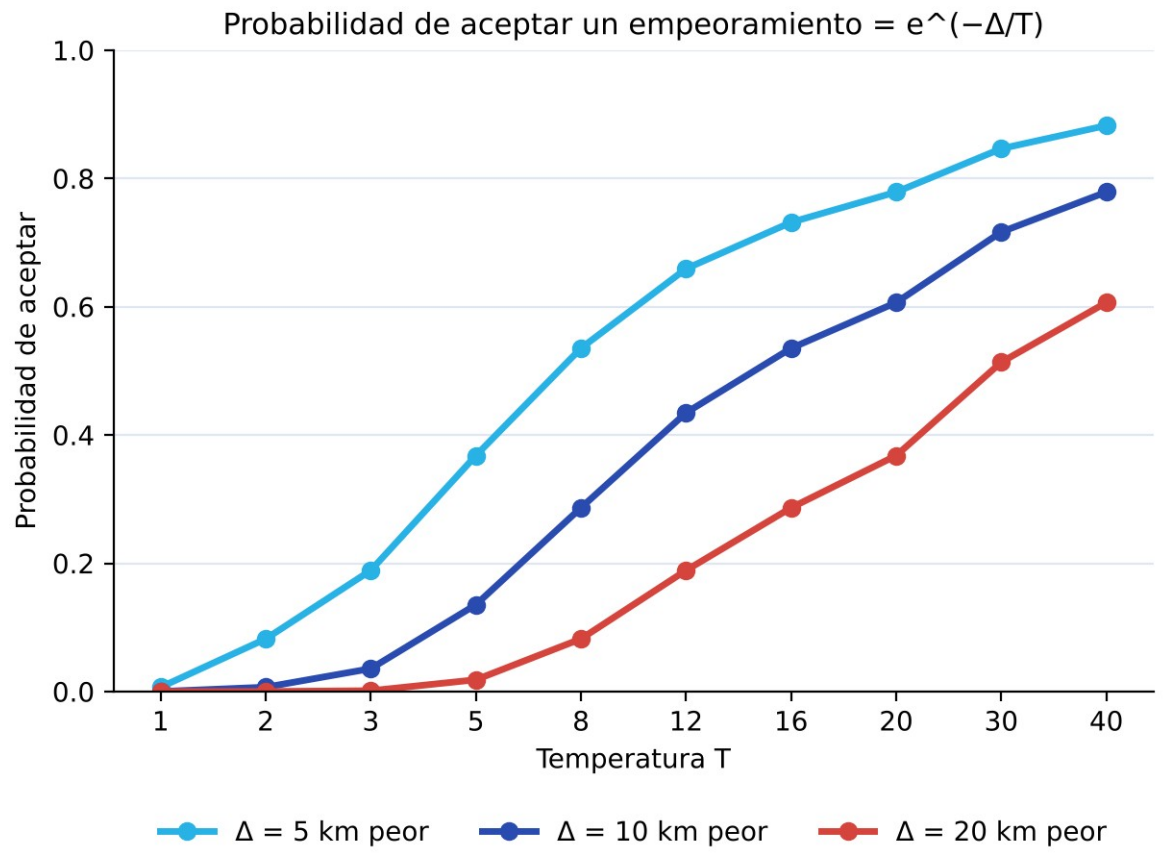
En palabras

- Si el vecino mejora, se acepta siempre.
- Si empeora en Δ, se acepta con probabilidad e^(−Δ/T).
- T alta: acepta casi todo (explora). T baja: solo acepta mejoras.

Parámetros

- Temperatura inicial T_0 y mínima
- Factor de enfriamiento α
- Repeticiones L por temperatura

Recocido simulado: aceptar peores soluciones para escapar



Cómo funciona

1. Partir de una solución y una temperatura alta T.

2. Probar un movimiento (p. ej. 2-opt).

3. Si mejora, aceptarlo. Si empeora Δ, aceptarlo con probabilidad e^(−Δ/T).

4. Enfriar (T ← 0,95·T) y repetir.

Con T alta explora casi sin restricciones; con T baja se comporta como mejora local.

Algoritmo genético: pseudocódigo

```
P ← N permutaciones aleatorias de los clientes
repetir por G generaciones:
    evaluar costo de cada individuo
        (cortar la permutación por capacidad Q)
    hijos ← ∅
    mientras |hijos| < N:
        A, B ← dos padres por torneo
        h ← cruce OX(A, B)
        con prob. p_m: mutar h (intercambio)
        agregar h a hijos
    P ← los N mejores de P ∪ hijos     (elitismo)
devolver el mejor de P
```

En palabras

- Cada individuo es un orden de visita de los clientes.
- Los padres de menor costo se cruzan para producir hijos.
- La mutación mantiene la diversidad; el elitismo conserva lo mejor.

Parámetros

- Tamaño de población N
- Generaciones G
- Probabilidad de mutación p_m

Colonia de hormigas: pseudocódigo

```
τ[i][j] ← τ₀ para todo arco; η[i][j] ← 1 / c[i][j]
repetir por iteraciones:
    para cada hormiga h:
        construir una ruta; desde i elegir j no visitado
        con probabilidad
          p[i][j] = τ[i][j]^α · η[i][j]^β
                    / Σ τ[i][l]^α · η[i][l]^β
    evaporar: τ[i][j] ← (1 − ρ) · τ[i][j]
    depositar: para cada arco (i, j) de la ruta de h:
        τ[i][j] ← τ[i][j] + Q / costo(ruta_h)
devolver la mejor ruta encontrada
```

En palabras

- Las hormigas construyen rutas eligiendo arcos con probabilidad.
- La probabilidad crece con la feromona τ y con la cercanía η = 1/c.
- Las rutas cortas dejan más feromona; la evaporación olvida las malas.

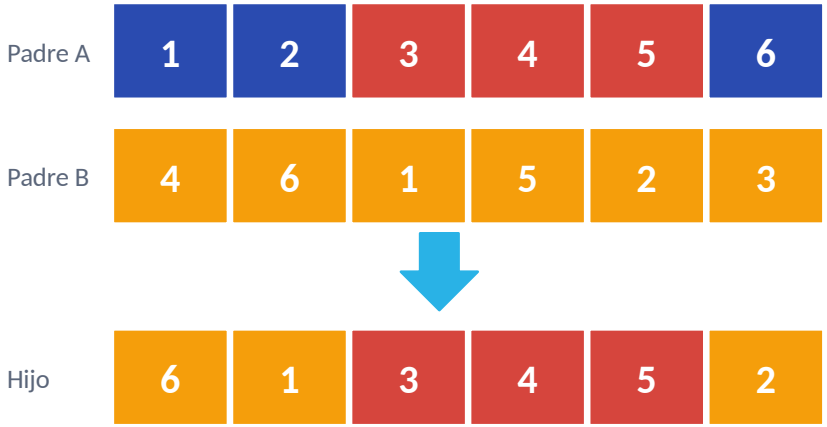
Parámetros

- α (peso de la feromona), β (peso de la cercanía)
- Evaporación ρ y número de hormigas

Genético y colonia de hormigas en el VRP

Algoritmo genético

Cromosoma = orden de visita (permutación). Cruce OX:



Se copia un tramo de A (3,4,5) y el resto se completa con el orden de B. Luego: mutación (intercambio) y selección.

Colonia de hormigas (ACO)

Probabilidad de ir de i a j:

$$p_{ij} = \tau_{ij}^{\alpha} \cdot \eta_{ij}^{\beta} \,/\, \Sigma_l \, \tau_{il}^{\alpha} \cdot \eta_{il}^{\beta}$$

$$\eta_{ij} = 1 \,/\, c_{ij} \quad \text{(visibilidad)}$$

Actualización de feromona:

$$\tau_{ij} \leftarrow (1 - \rho)\, \tau_{ij} + \Sigma_a \, \Delta\tau_{ij}^{a}$$

$$\Delta\tau = Q \,/\, (\text{costo de la ruta de } a)$$

- Las rutas cortas acumulan más feromona
- La evaporación ρ olvida malos caminos
- Hormigas = vehículos construyendo rutas

GRASP y VNS: pseudocódigo

GRASP

```
repetir N veces:
    s ← construcción voraz aleatorizada:
        en cada paso elegir al azar entre los
        mejores candidatos (lista RCL)
    s ← búsqueda local(s)
    si costo(s) < costo(mejor): mejor ← s
devolver mejor
```

VNS (búsqueda de vecindario variable)

```
s ← inicial; k ← 1
mientras k ≤ k_max:
    s' ← aleatoria en el vecindario N_k(s)
    s" ← búsqueda local(s')
    si costo(s") < costo(s):
        s ← s"; k ← 1
    si no: k ← k + 1
devolver s
```

GRASP: construye muchas soluciones voraces con azar y las mejora con búsqueda local. **VNS:** cuando la búsqueda local se estanca, cambia de tipo de vecindario (más grande) y vuelve a intentar. **ALNS:** destruye parte de la solución y la repara; es el estado del arte en VRP con muchas restricciones.

Branch and bound (exacto): pseudocódigo

```
lista ← { problema original }; cota_sup ← ∞
mientras lista ≠ ∅:
    tomar un subproblema P de la lista
    z ← valor de la relajación lineal de P
    si P es infactible o z ≥ cota_sup: descartar (poda)
    si no, si la solución de P es entera:
        cota_sup ← z; guardar la solución
    si no:
        elegir una variable fraccionaria x_k
        crear P con x_k = 0 y P con x_k = 1
devolver la mejor solución entera
```

En palabras

- Resuelve el problema relajado (variables continuas) y ramifica donde sale una fracción.
- Poda las ramas que no pueden mejorar la mejor solución entera conocida.
- Es lo que hace el solver (HiGHS) con las fórmulas MTZ del curso.

En la herramienta

- La brecha (gap) = (mejor solución − mejor cota) / mejor solución
- Gap 0 % = óptimo probado

Métodos exactos y matheurísticas

E Exactos

- Branch & bound con las formulaciones vistas (MTZ, flujo por vehículo)
- Branch & cut: se agregan los cortes de sub-tours violados
- Branch & price: generación de columnas sobre rutas
- Garantizan optimalidad; escalan poco

H Matheurísticas (híbridos)

- Usan el solver MILP dentro de una búsqueda
- LNS: se fija parte de la solución y se resuelve el resto exacto
- Relax-and-fix y fijación por costo reducido
- Calidad de la heurística con rigor del modelo

Qué hay que ajustar en cada metaheurística

Método	Parámetros principales	Efecto
Búsqueda tabú	Tamaño de la lista, iteraciones	Lista larga: explora más; corta: se repite
Recocido simulado	Temperatura inicial T, factor de enfriamiento	T alta: acepta casi todo; baja: mejora local
Algoritmo genético	Población, cruce, mutación, generaciones	Más población: más diversidad y más tiempo
Colonia de hormigas	α, β, evaporación ρ, hormigas	α sube la memoria; β sube la visibilidad 1/c
VNS / ALNS	Vecindarios, % de destrucción	Cambiar de estructura de búsqueda al estancarse

Regla práctica: empezar con valores de la literatura y ajustar con pruebas sobre instancias parecidas a las reales.

¿Cuándo usar cada enfoque?

	Exacto (MILP)	Heurística	Metaheurística
Calidad	Óptima probada	Aceptable	Muy buena
Tiempo	Crece rápido	Segundos	Segundos a minutos
Tamaño	Decenas a pocos cientos	Miles	Miles
Cambiar restricciones	Agregar ecuaciones	Reprogramar	Reprogramar operadores
Mejor para	Decisiones estratégicas y validar	Solución inicial rápida	Operación diaria a gran escala

Regla práctica: pocos clientes y necesidad de garantías → modelo exacto; muchos clientes → metaheurística; en la práctica, heurística para empezar, metaheurística para mejorar y modelo exacto para validar.

¿Qué método elijo? Guía de decisión

¿Pocos clientes (≲ 30–50)? Sí → **¿Hace falta probar el óptimo?** Sí → **Modelo exacto (MILP, MTZ / DFJ)**

No ↓

Heurística constructiva + mejora local

Rápida; sirve de solución inicial.

Metaheurística (tabú, recocido, genético)

Calidad alta en miles de clientes.

Matheurística (LNS con MILP)

Rigor del modelo y escala de la heurística.

Aplicación real · planificación según el tamaño

Una tienda con 25 puntos usa el modelo exacto; una distribuidora con 2 000 pedidos diarios combina heurística y metaheurística.

Tres formas de resolver cada variante

Modelo matemático

- Formulación MILP (MTZ / flujo por vehículo)
- Resuelto con HiGHS
- Da la solución óptima o la mejor con su cota

Garantía: gap = 0 % = óptimo probado

Heurística

- Vecino más cercano
- Ahorros (Clarke-Wright)
- Barrido
- Inserción más barata
- Constructiva + mejora local (VND)

Rápida: milisegundos; sin garantía

Metaheurística

- Recocido simulado
- Búsqueda tabú
- Algoritmo genético
- Colonia de hormigas
- GRASP
- VNS

Mejor calidad; tiempo configurable

Brecha (gap) de una heurística o metaheurística = `(costo obtenido − mejor cota) / costo obtenido`, donde la cota inferior la calcula el modelo matemático (rama y cota).

11 algoritmos de solución implementados

Algoritmo	Tipo	Idea
Vecino más cercano	Constructiva	Va al cliente factible más cercano
Ahorros (Clarke-Wright)	Constructiva	Une rutas por mayor ahorro $s = c_{0i} + c_{0j} - c_{ij}$
Barrido	Constructiva	Agrupa por ángulo respecto al depósito
Inserción más barata	Constructiva	Inserta donde menos aumenta el costo
Constructiva + VND	Mejora local	Relocate, swap y 2-opt hasta no mejorar
Recocido simulado	Metaheurística	Acepta peores con prob. e^(−Δ/T); enfría T
Búsqueda tabú	Metaheurística	Mejor vecino no prohibido + aspiración
Algoritmo genético	Metaheurística	Población, cruce OX, mutación, elitismo
Colonia de hormigas	Metaheurística	Feromona τ y cercanía η guían la construcción
GRASP	Metaheurística	Voraz aleatorizado + búsqueda local, repetido
VNS	Metaheurística	Agitación y cambio de vecindario

Prueba de las heurísticas y metaheurísticas

Algoritmo	Tipo	En el óptimo	Gap mediano	Factibles
Vecino más cercano	Heur.	6 de 26	15.31 %	26 de 26
Ahorros (Clarke-Wright)	Heur.	10 de 26	4.68 %	26 de 26
Barrido (sweep)	Heur.	1 de 26	26.73 %	19 de 26
Inserción más barata	Heur.	8 de 26	3.15 %	26 de 26
Constructiva + mejora local (VND)	Heur.	20 de 26	0.00 %	26 de 26
Recocido simulado	Meta	25 de 26	0.00 %	26 de 26
Búsqueda tabú	Meta	25 de 26	0.00 %	26 de 26
Algoritmo genético	Meta	25 de 26	0.00 %	26 de 26
Colonia de hormigas	Meta	25 de 26	0.00 %	26 de 26
GRASP	Meta	20 de 26	0.00 %	26 de 26
VNS (búsqueda de vecindario variable)	Meta	26 de 26	0.00 %	26 de 26

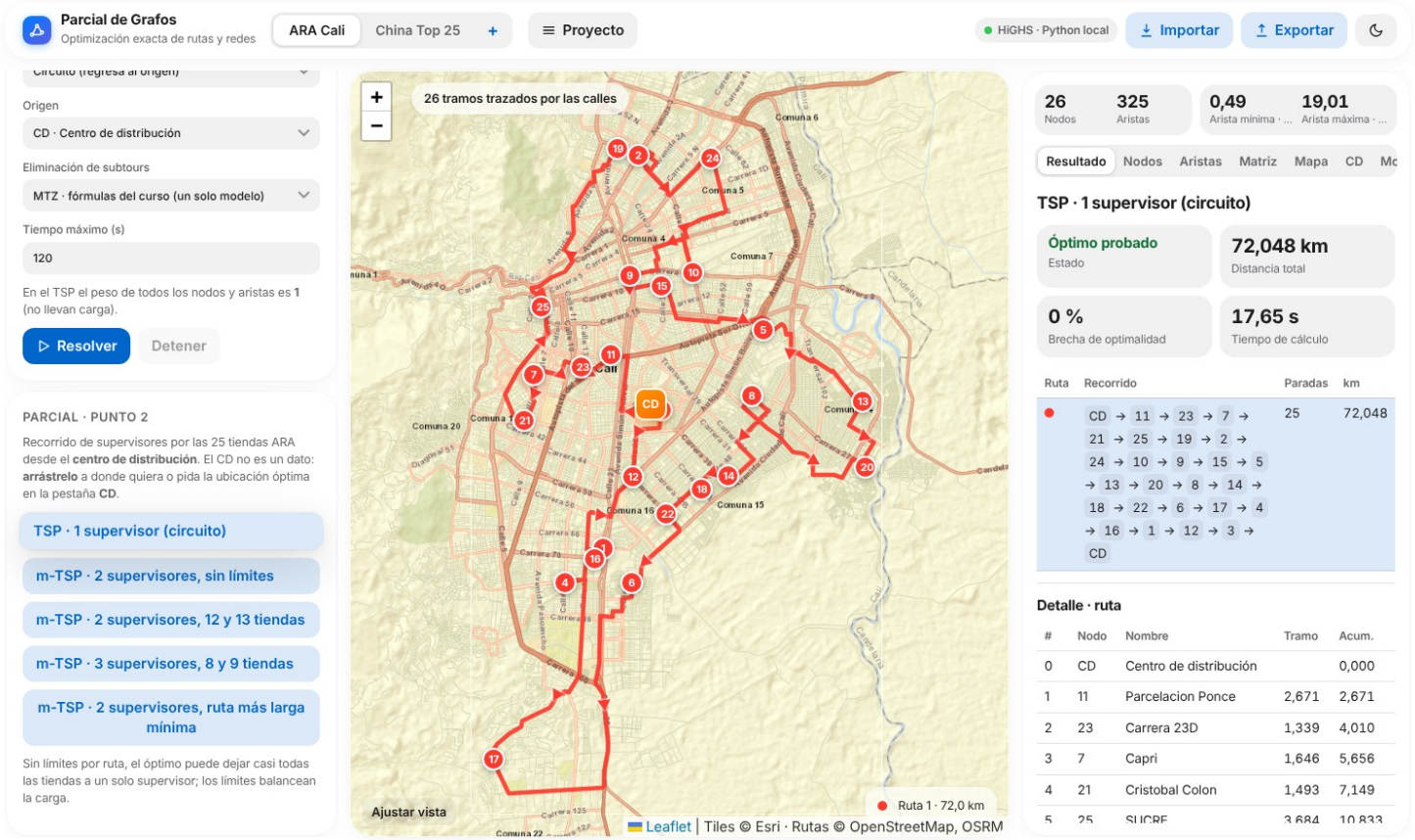
Sobre las 26 variantes con instancia de 7 nodos (excepto SDVRP). Tiempo: 1 s para las constructivas y 1,2 s para las metaheurísticas.

D

Ejemplos resueltos con optimización exacta

Las formulaciones del curso aplicadas a 25 tiendas ARA en Cali, resueltas con una herramienta propia (HiGHS).

Ejemplo 1 · TSP: un supervisor visita 25 tiendas



72,048 km

distancia óptima del circuito

26 nodos

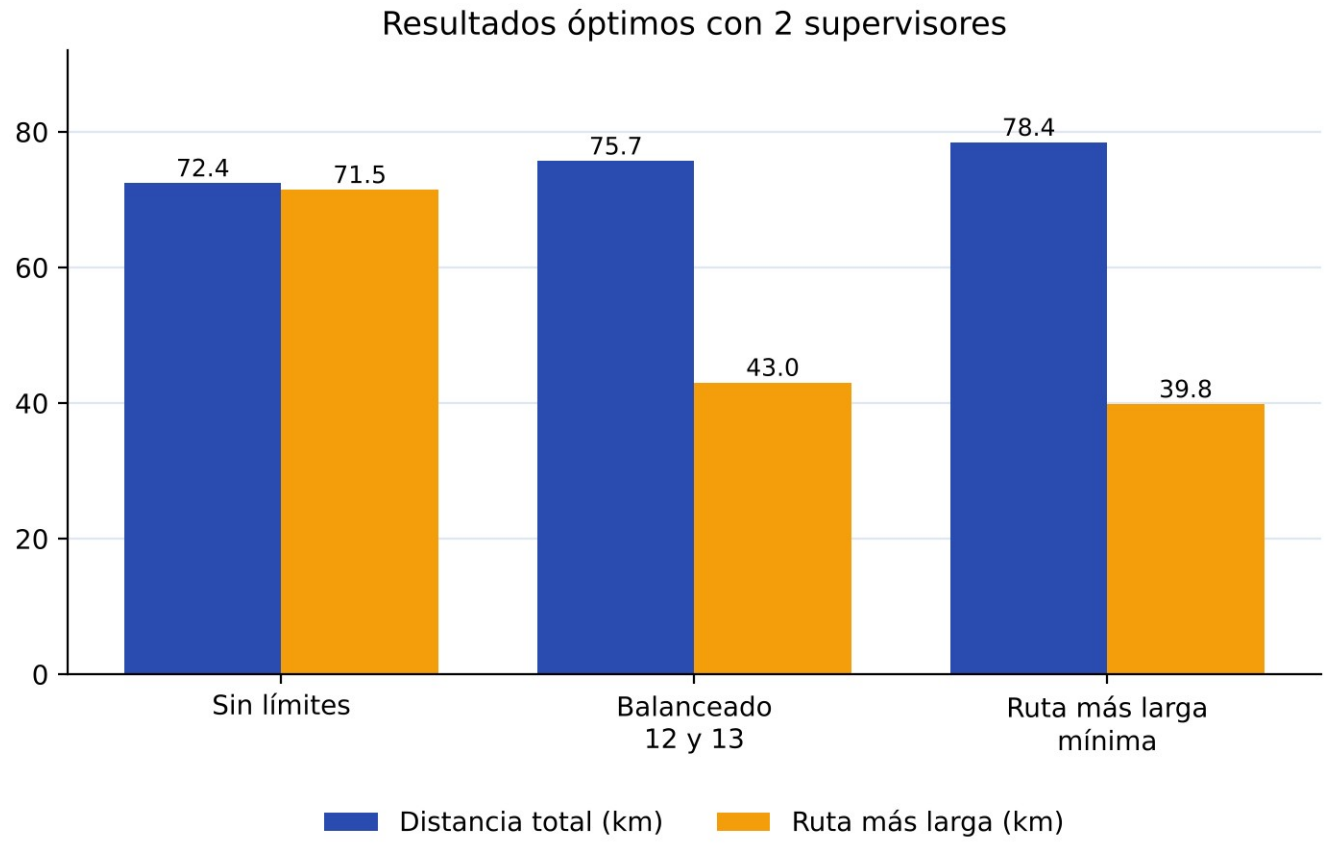
centro de distribución + 25 tiendas

Óptimo probado

con las ecuaciones (1)–(3) del curso (MTZ)

Ruta dibujada sobre las calles reales (OpenStreetMap). Distancias por carretera.

Ejemplo 2 · mTSP: dos supervisores, tres formas de repartir



Lectura

- Sin límites, un supervisor visita 24 tiendas y el otro 1: barato en total, injusto.
- Balanceado (12 y 13) cuesta 3,2 km más y reparte bien.
- Minimizar la ruta más larga baja la peor jornada de 71 a 40 km.

Ejemplo 3 · flota mixta y ventanas de tiempo

Flota heterogénea (HFVRP)

Vehículo	Capacidad	Costo/km	km/h
Camión de reparto ×2	5	1,0	30
Tractocamión ×1	34	2,4	50

Con 14 clientes de peso 1, el óptimo usa los dos camiones llenos (5/5) y un tractocamión con 4/34: costo total 104,97.

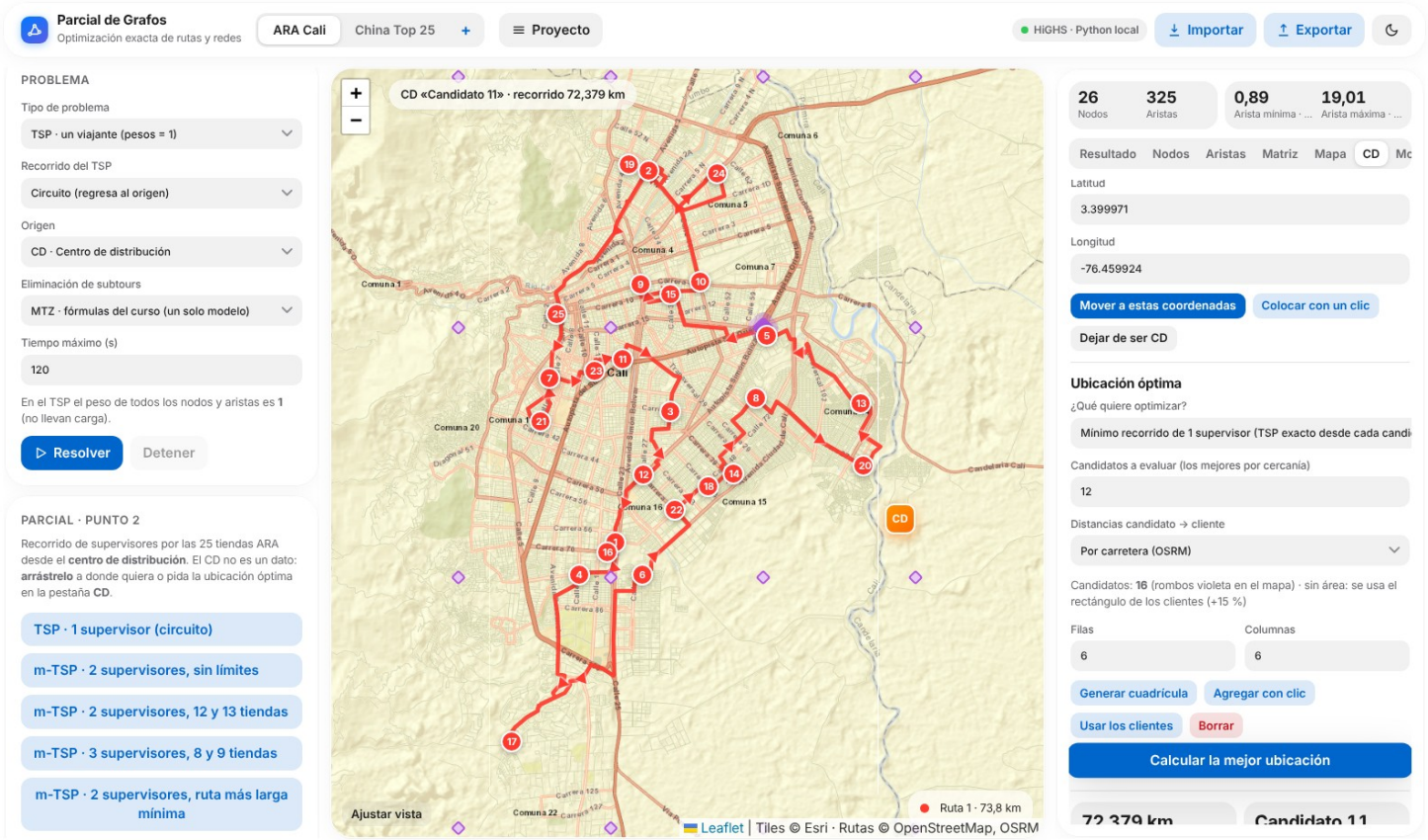
El modelo decide qué vehículo atiende cada ruta según capacidad y costo por km.

Ventanas de tiempo (VRPTW)

- Franja horaria de cada cliente que se marca y desmarca
- Hora de salida y tiempo de servicio configurables
- El resultado muestra la hora de llegada a cada cliente
- Rutas abiertas y balanceadas combinables

Comprobado: todas las llegadas quedaron dentro de su franja y ninguna carga superó la capacidad.

Ejemplo 4 · LRP: ¿dónde poner el centro de distribución?



93,6 → 72,4 km

recorrido TSP: CD mal ubicado frente al óptimo

–22,7 %

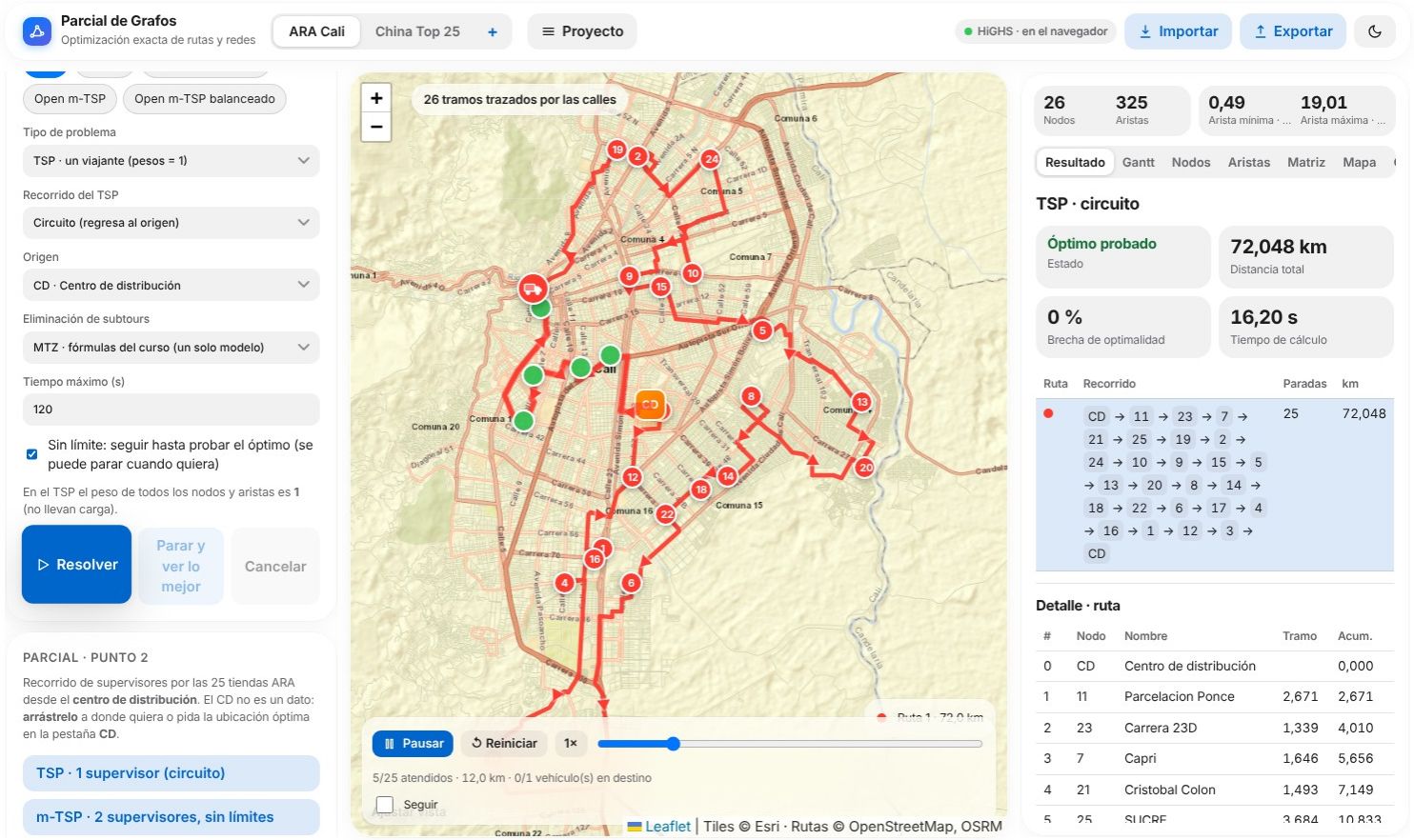
ahorro por elegir bien la ubicación

TSP exacto

se resuelve en cada candidato (rombos violeta)

El CD no es un dato: se arrastra donde se quiera o se pide la ubicación óptima entre los candidatos.

La herramienta: resolver, medir el gap y simular



TSP de 25 tiendas en Cali: óptimo probado (brecha 0 %), con la simulación de vehículos.

Brecha en vivo

mejor solución, cota y gap mientras resuelve

Parar cuando se quiera

muestra la mejor solución con su brecha

Simulación y Gantt

vehículos por las calles y tiempos por ruta

Comparación en Cali: CVRP con 25 tiendas

Método	Tipo	Costo (km)	Gap	Tiempo (s)
Modelo matemático (límite 8 s)	Exacto	87,83	21,37 %	8,0
Vecino más cercano	Heurística	101,93	32,24 %	0,0
Ahorros (Clarke-Wright)	Heurística	76,56	9,78 %	0,0
Constructiva + VND	Heurística	87,05	20,66 %	0,0
Recocido simulado	Meta	75,02	7,93 %	1,7
Búsqueda tabú	Meta	75,02	7,93 %	1,7
Algoritmo genético	Meta	74,87	7,75 %	1,7
Colonia de hormigas	Meta	74,87	7,75 %	1,7
GRASP · VNS	Meta	74,87	7,75 %	1,7

Lectura

- Mejor cota del modelo: 69,07 km.
- Seis métodos llegan a 74,87–75,02 km: brecha 7,8 %.
- Con solo 8 s, el modelo exacto no supera a las metaheurísticas.

Gap = (costo − cota) / costo. La cota (69,07) es la del modelo matemático en 8 s; con más tiempo sube y el gap baja.

Conclusiones

1

Un mismo núcleo

Todas las variantes agregan restricciones o variables a x_{ijk}, c_{ij} y sub-tours.

2

Modelos pequeños, aplicaciones grandes

Abierto, distancia, flota, depósitos y localización con pocas ecuaciones nuevas.

3

Exacto vs. heurístico

El exacto da el óptimo pero escala poco; las metaheurísticas escalan con buena calidad.

4

Lo habitual

Heurística para empezar, metaheurística para mejorar, modelo exacto para validar.

APA 7

Referencias

Acevedo-Chedid, J. (2025). Capítulo 4: Ruteo de vehículos [Diapositivas]. Universidad Tecnológica de Bolívar.

Clarke, G., & Wright, J. W. (1964). Scheduling of vehicles from a central depot to a number of delivery points. Operations Research, 12(4), 568–581.

Dantzig, G. B., & Ramser, J. H. (1959). The truck dispatching problem. Management Science, 6(1), 80–91.

Glover, F. (1986). Future paths for integer programming and links to artificial intelligence. Computers & Operations Research, 13(5), 533–549.

Laporte, G. (1992). The vehicle routing problem: An overview of exact and approximate algorithms. European Journal of Operational Research, 59(3), 345–358.

Miller, C. E., Tucker, A. W., & Zemlin, R. A. (1960). Integer programming formulation of traveling salesman problems. Journal of the ACM, 7(4), 326–329.

Ropke, S., & Pisinger, D. (2006). An adaptive large neighborhood search heuristic for the pickup and delivery problem with time windows. Transportation Science, 40(4), 455–472.

Toth, P., & Vigo, D. (Eds.). (2014). Vehicle routing: Problems, methods, and applications (2.ª ed.). SIAM.